Module-4

Software Project Management

A Project Management Framework

What is a project management framework? Basically, it is a combination of processes, tasks,and tools used to transition a project from start to finish. An overview of a generic process used by this framework is:

- **Initiation**: when the project starts
- **Planning**: when all of the key decisions are made
- **Execution**: when project work actually takes place
- **Control**: when adjustments are made to the plan
- **Monitoring**: when project progress is checked
- **Termination**: when the project comes to an end

Each stage of this process involves the completion of many tasks by project team members using various tools. The generic process just described is a project's lifecycle from initiation to termination, which is one part of the framework. Now let's dive deeper into the **three parts of a project management framework**: lifecycle, control cycle, and tools and templates.

Lifecycle

The **lifecycle** of the framework explains the stages involved in the project and what needs to happen at each stage. It allows the management team to make adjustments and customize thestages based on the size and scope of the project.

For example, a company that is completing a project designing children's learning materialsmay have a lifecycle as follows:

1. First, the company initiates the project. The company may go through numerous upfront procedures to decide whether or not to pursue the business opportunity, make it available forbids, and finalize contracts.

2. Second, the company plans the project. The company defines in detail the projectdeliverables, schedule, detailed budget, etc.

3. Third, the company executes the project by proceeding with actual project tasks such as creating the learning materials. It manages the project according to plan by using the controlcycle to monitor progress and adjust the plan if required.

4. Finally, when the materials are completed, the company terminates the project and followsa closure procedure.

Resource management

All elements used to develop a software product may be assumed as resource for that project. This may include human resource, productive tools and software libraries. The resources are available in limited quantity and stay in the organization as a pool of assets. The shortage of resources hampers the development of project and it can lag behind the schedule. Allocating extra resources increases development cost in the end. It is therefore necessary to estimate and allocate adequate resources for the project.

Resource management includes -

- Defining proper organization project by creating a project team and allocating responsibilities to each team member
- Determining resources required at a particular stage and their availability
- Manage Resources by generating resource request when they are required and de-allocating them when they are no more needed.

Methods to estimate project time and cost

- Quality time and cost estimates are the bedrock of project control. Past experience is the best starting point for these estimates. The quality of estimates is influenced by other factors such as people, technology, and downtimes. The key for getting estimates that represent realistic average times and costs is to have an organization culture that allows errors in estimates without incriminations. If times represent average time, we should expect that 50 percent will be less than the estimate and 50 percent will exceed the estimate. The use of teams that are highly motivated can help in keeping task times and costs near the average. For this reason, it is crucial to get theteam to buy into time and cost estimates.
- Using top-down estimates is good for initial and strategic decision making or in situations where the costs associated with developing better estimates have little benefit. However, in most cases the bottom-up approach to estimating is preferred and more reliable because it assesses each work package, rather than the whole project, section, or deliverable of a project. Estimating time and costs for each work package facilitates development of the project schedule and a time-phased budget, which are needed to control the project as it is implemented. Using the

estimating guidelines will help eliminate many common mistakes made by those unacquainted with estimating times and costs for project control. Establishing a time and cost estimating database fits well with the learning organization philosophy.

- The level of time and cost detail should follow the old saying of "no more than is necessary and sufficient." Managers must remember to differentiate between committed outlays, actual costs, and scheduled costs. It is well known that upfront efforts in clearly defining project objectives, scope, and specifications vastly improve time and cost estimate accuracy.
- Finally, how estimates are gathered and how they are used can affect their usefulness for planning and control. The team climate, organization culture, and organization structure can strongly influence the importance attached to time and cost estimates and how they are used in managing projects.

Risk

Risk is an expectation of loss, a potential problem that may or may not occur in the future. It is generally caused due to lack of information, control or time.A possibility of suffering from loss in software development process is called a software risk. Loss can be anything, increase in production cost, development of poor quality software, not being able to complete the project on time. Software risk exists because the future is uncertain and there are many known and unknown things that cannot be incorporated in the project plan. A software risk can be of two types (a) internal risks that are within the control of the project manager and (2) external risks that are beyond the control of project manager. Risk management is carried out to:

1. Identify the risk
2. Reduce the impact of risk
3. Reduce the probability or likelihood of risk
4. Risk monitoring

A project manager has to deal with risks arising from three possible cases:

1. Known knowns are software risks that are actually facts known to the team as well as to the entire project. For example not having enough number of developers can delay the project delivery. Such risks are described and included in the Project Management Plan.

2. Known unknowns are risks that the project team is aware of but it is unknown that such risk exists in the project or not. For example if the communication with the

client is not of good level then it is not possible to capture the requirement properly. This is a fact known to the project team however whether the client has communicated all the information properly or not is unknown to the project.
3. Unknown Unknowns are those kind of risks about which the organization has no idea. Such risks are generally related to technology such as working with technologies or tools that you have no idea about because your client wants you to work that way suddenly exposes you to absolutely unknown unknown risks.

Software risk management is all about risk quantification of risk. This includes:
1. Giving a precise description of risk event that can occur in the project
2. Defining risk probability that would explain what are the chances for that risk to occur
3. Defining How much loss a particular risk can cause
4. Defining the liability potential of risk

Risk Management comprises of following processes:
1. Software Risk Identification
2. Software Risk Analysis
3. Software Risk Planning
4. Software Risk Monitoring

RISK IDENTIFICATION

Risk identification is a systematic attempt to specify threats to the project plan (estimates,

schedule, resource loading, etc.). By identifying known and predictable risks, the project manager takes a first step toward avoiding them when possible and controlling them when necessary.

There are two distinct types of risks: generic risks and product-specific risks. *Generic risks* are a potential threat to every software project. *Product-specific risks* can be identified only by those with a clear understanding of the technology, the people, and the environment that isspecific to the project at hand.

One method for identifying risks is to create a *risk item checklist.* The checklist can be used for risk identification and focuses on some subset of known and predictable

risks in the following generic subcategories:

- *Product size*—risks associated with the overall size of the software to be built or modified.
- *Business impact*—risks associated with constraints imposed by management or themarketplace.
- *Customer characteristics*—risks associated with the sophistication of the customer and thedeveloper's ability to communicate with the customer in a timely manner.
- *Process definition*—risks associated with the degree to which the software process has beendefined and is followed by the development organization.
- *Development environment*—risks associated with the availability and quality of the tools tobe used to build the product.
- *Technology to be built*—risks associated with the complexity of the system to be built andthe "newness" of the technology that is packaged by the system.
- *Staff size and experience*—risks associated with the overall technical and project experienceof the software engineers who will do the work.

6.3.1 Assessing Overall Project Risk

The following questions have derived from risk data obtained by surveying experienced software project managers in different part of the world [KEI98]. The questions are ordered by their relative importance to the success of a project.

1. Have top software and customer managers formally committed to support the project?
2. Are end-users enthusiastically committed to the project and the system/product to be built?
3. Are requirements fully understood by the software engineering team and their customers?
4. Have customers been involved fully in the definition of requirements?
5. Do end-users have realistic expectations?
6. Is project scope stable?
7. Does the software engineering team have the right mix of skills?
8. Are project requirements stable?
9. Does the project team have experience with the technology to be implemented?

The risk components are defined in the following manner:

- *Performance risk*—the degree of uncertainty that the product will meet its requirements andbe fit for its intended use.
- *Cost risk*—the degree of uncertainty that the project budget will be maintained.
- *Support risk*—the degree of uncertainty that the resultant software will be easy to correct,adapt, and enhance.
- *Schedule risk*—the degree of uncertainty that the project schedule will be maintained andthat the product will be delivered on time.

Software Risk Analysis

Software Risk analysisis a very important aspect of risk management. In this phase the risk is identified and then categorized. After the categorization of risk, the level, likelihood (percentage) and impact of the risk is analyzed. Likelihood is defined in percentage after examining what are the chances of risk to occur due to various technical conditions. These technical conditions can be:

1. Complexity of the technology
2. Technical knowledge possessed by the testing team
3. Conflicts within the team
4. Teams being distributed over a large geographical area
5. Usage of poor quality testing tools

With impact we mean the consequence of a risk in case it happens. It is important to know about the impact because it is necessary to know how a business can get affected:

1. What will be the loss to the customer
2. How would the business suffer
3. Loss of reputation or harm to society
4. Monetary losses
5. Legal actions against the company
6. Cancellation of business license

Level of risk is identified with the help of:

Qualitative Risk Analysis: Here you define risk as:

- High
- Low
- Medium

Quantitative Risk Analysis: can be used for software risk analysis but is

considered inappropriate because risk level is defined in % which does not give a very clear picture.

Project Management Concepts

The main goal of software project management is to enable a group of software engineers to work efficiently towards successful completion of the project. The management of software development is dependent on four factors:

- The People
- The Product
- The Process
- The Project

Project Management

There are many software engineers involved in the development of a software product. The primary job of the project manager is to ensure that the project is completed within budget and on schedule.

Job Responsibilities of a Software Project Manager

- Software managers are responsible for planning and scheduling project development. Manager must decide what objectives are to be achieved, what resources are required to achieve the objectives, how and when the resources are to be acquired and how the goals are to be achieved.
- Software managers takes responsibility for project proposal writing, project cost estimation, project staffing, project monitoring and control, software configuration management, risk management, interfacing with clients, managerial report writing and presentation.
- Software managers monitor progress to check that the development is on time and within budget.

Skills Necessary for Software Project Management

- Good qualitative judgment and decision-making capabilities
- Good knowledge of latest software project management techniques such as cost estimation, risk management, configuration management.
- Good communication skill and previous experience in managing similar

projects.

Project Size Estimation Metrics, Line Of Control (LOC) and Function Point Metric (FP)

The size of a project is obviously not the number of bytes that the source code occupies. The project size is a measure of the problem complexity in terms of the effort and time required to develop the product.
Two metrics are widely used to estimate size:

- Lines of Code (LOC)
- Function Point (FP)

Lines Of Code (LOC)

LOC can be defined as the number of delivered lines of code in software excluding the comments and blank lines. LOC depends on the programming language chosen for the project. The exact number of the lines of code can only be determined after the project is complete since less information about the project is available at the early stage of development.
In order to estimate the LOC count at the beginning of a project, project managers usually divide the problem into modules and each modules into sub modules and a so on until the sizes of the different leaf level modules can be approximately predicted.
Disadvantages:

- LOC is language dependent. A line of assembler is not the same as a line of COBOL.
- LOC measure correlates poorly with the quality and efficiency of the code. A larger code size does not necessary imply better quality or higher efficiency.
- LOC metrics penalizes use of higher level programming languages, code reuse etc.
- It is very difficult to accurately estimate LOC in the final product from the problem specification. The LOC count can be accurately computed only after the code has been fully developed.

Function Point Metric

- Function Points measure software size by quantifying the functionality

provided to the user based solely on logical design and functional specifications

- Function point analysis is a method of quantifying the size and complexity of a software system in terms of the functions that the system delivers to the user
- It is independent of the computer language, development methodology, technology or capability of the project team used to develop the application.
- Function point analysis is designed to measure business applications (not scientific applications) .
- Function points are independent of the language, tools, or methodologies used for implementation
- Function points can be estimated early in analysis and design
- Since function points are based on the system user's external view of the system, non-technical users of the software system have a better understanding of what function points are measuring.

Objectives of Function Point Counting

- Measure functionality that the user requests and receives
- Measure software development and maintenance independently of technology used for implementation

Steps of Function Point Counting

- Determine the type of function point count
- Identify the counting scope and application boundary
- Determine the Unadjusted Function Point Count
- Count Data Functions
- Count Transactional Functions
- Determine the Value Adjustment Factor
- Calculate the Adjusted Function Point Count

Function point metric estimates the size of a software product directly from the problem specification.

The different parameters are:

- Number Of Inputs:

Each data item input by the user is counted.

- Number Of Outputs:

The outputs refers to reports printed, screen outputs, error messages produced etc.

- Number Of Inquiries:

It is the number of distinct interactive queries which can be made by the users.

- Number Of Files:

Each logical file is counted. A logical file means groups of logically related data. Thus logical files can be data structures or physical files.

- Number Of Interfaces:

Here the interfaces which are used to exchange information with other systems. Examples of interfaces are data files on tapes, disks, communication links with other systems etc.

Project Estimation Techniques

The estimation of various project parameters is a basic project planning activity. The project parameters that are estimated include:

- Project size(i.e. size estimation)
- Project duration
- Effort required to develop the software

There are three broad categories of estimation techniques:

- Empirical estimation techniques
- Heuristic techniques
- Analytical estimation techniques

Empirical Estimation Techniques

Empirical estimation techniques are based on making an educated guess of the project parameters. While using this technique, prior experience with the development of similar products is useful.

Heuristic Techniques

Heuristic techniques assume that the relationships among the different project parameters can be modelled using suitable mathematical expressions. Once the basic (independent) parameters are known, the other (dependent) parameters can be easily determined by substituting the value of the basic parameters in the mathematical expression. Different heuristic estimation models can be divided into two categories:

- Single variable model
- Multivariable model

A single variable estimation model takes the following form: Estimated parameter = c1* ed1

Where e is a characteristics of the software, c1 and d1 are constants. A multivariable

cost estimation model takes the following form: Estimated Resource = c1 * e d1 + c * e d2 + ...
Where e1, e2 are the basic characteristics of the software.
c1, c2, d1, d2 are constants.

Analytical Estimation Techniques

Analytical estimation techniques derive the required results starting with certain basic assumptions regarding the project. This technique does have a scientific basis. Halstead's Software Science an Analytical Estimation Techniques

Halstead's software science is an analytical technique to measure size, development effort, and development cost of software products. Halstead used a few primitive program parameters to develop the expressions for the overall program length, potential minimum volume, language level, and development time.
For a given program, let:

- η1 be the number of unique operators used in the program
- η2 be the number of unique operands used in the program
- N1 be the total number of operators used in the program
- N2 be the total number of operands used in the program.

There is no general agreement among researchers on what is the most meaningful way to define the operators and operands for different programming languages. For instance, assignment, arithmetic, and logical operators are usually counted as operators. A pair of parentheses, as well as a block begin and block end pair, are considered as single operators.
The constructs if.....then.....else.....endif and a while do are treated as single operators. A sequence operator ';' is treated as a single operator. Empirical Estimation Techniques

Cost estimation is a part of the planning stage of any engineering activity. For any new software project, it is necessary to know how much it will cost to develop and how much development time it will take. Cost in a project is due to the requirements for software, hardware and human resources. Hardware resources such as computer time, terminal time and memory required for the project, software resources include the tools and compilers needed during development.

Cost estimates can be made either top-down or bottom-up. Top-down estimation

first focuses on system level costs such as the computing resources and personal required to develop the system, quality assurance, system integration, training. Bottom-up cost estimation first estimates the cost to develop each module or subsystem. Those costs are combined to arrive at an overall estimate. Two popular empirical estimation techniques are:

Expert Judgment Technique

The most widely used cost estimation technique is the expert judgment, which is an inherently top-down estimation technique. In this approach an expert makes an educated guess of the problem size after analyzing the problem thoroughly. The expert estimates the cost of the different modules or subsystems and then combines them to arrive at the overall estimate. However, this technique is subject to human errors and individual bias. An expert making an estimate may not have experience and knowledge of all aspects of a project. The advantage of expert judgment is the estimation made by a group of experts. Estimation by a group of experts minimizes factors such as lack of familiarity with a particular aspect of a project, personal bias.

Delphi Cost Estimation

Delphi cost estimation approach tries to overcome some of the short comings of the expert judgment approach. Delphi estimation is carried out by a team consisting of a group of experts and a coordinator. The Delphi technique can be adapted to software cost estimation in the following manner:

- A coordinator provides each estimator with the software requirement specification (SRS) document and a form for recording a cost estimate.
- Estimators study the definition and complete their estimates anonymously and submit it to the coordinator. They may ask questions to the coordinator, but they do not discuss their estimates with one another.
- The coordinator prepares and distributes a summary of the estimator's responses and includes any unusual rationales noted by the estimators.
- Based on this summary, the estimators re-estimate. This process is iterated for several rounds. No group discussion is allowed during the entire process.

COCOMO: A Heuristic Estimation Technique

COCOMO was proposed by Boehm. Boehm postulated that any software development project can be classified into one of the following three categories

based on the development complexity: organic, semidetached, and embedded.

- Organic: In the organic mode the project deals with developing a well-understood application program. The size of the development team is reasonably small, and the team members are experienced in developing similar types of projects.
- Semidetached: In the semidetached mode the development team consists of a mixture of experienced and inexperienced staff. Team members may have limited experience on related systems but may be unfamiliar with some aspects of the system being developed.
- Embedded: In the embedded mode of software development, the project has tight constraints, which might be related to the target processor and its interface with the associated hardware.

Basic COCOMO

The basic COCOMO model gives an approximate estimate of the project parameters. The basic COCOMO estimation model is given by the following expressions:
Effort = a1 × (KLOC)a2 PM Tdev
= b1 × (Effort) b2 Months Where

(i) KLOC is the estimated size of the software product expressed in Kilo Lines of Code,
(ii) a1, a2, b1, b2 are constants for each category of software products,
(iii) Tdev is the estimated time to develop the software, expressed in months,
(iv) Effort is the total effort required to develop the software product, expressed in person months (PMs).

-